

生成式 AI 基礎：從 Transformer 到 LLM

M7 — Foundations of Generative AI

這一週不碰工具，只弄懂原理。搞清楚 LLM 到底在做什麼，
下三週在 Dify 裡設定的每一個欄位，你才知道自己在設什麼。

本講義大綱

01 技術演進與應用領域 · 從 2017 到 2026 的三個轉折

02 生成式 vs 判別式 · 跟前六週學的差在哪

03 四種生成式架構 · GAN · VAE · Diffusion · Transformer

04 Token 與 Tokenization · 模型眼中的文字

05 Attention 機制 · 一句話說完它在做什麼

06 三種 Transformer 架構 · 為什麼 RAG 要用兩種模型

07 Embedding 與語意空間 · 文字怎麼變成有距離的數字

08 預訓練 / 微調 / 推論 · 三件常被混為一談的事

09 讓模型懂你的領域 · 四種辦法與決策順序

10 推論參數 · Temperature、Top-p 與其他

11 幻覺的成因 · 為什麼它不是 bug

12 模型能力與規模 · 參數量、上下文、多模態

13 推理模型與多模態 · 2025 之後的進展

14 部署選項 · 雲端 vs Ollama 本地

15 Hugging Face 生態 · 模型從哪裡來

16 AI Studio 生成式範例 · 課堂實作

17 提示詞基礎與上下文工程 · 為什麼「怎麼問」差這麼多

18 倫理與資料落地 · 醫療場域的界線

這一週在整個課程的位置

你已經走過的路 (M1-M3)

- 資料工程 → 監督式學習 (迴歸、分類) → 非監督式學習 (分群、降維)
- 共同點：多半在既有資料上做預測、表示或分群，而不是生成新內容
- 評估通常有明確目標，例如 RMSE、F1 或分群穩定性

這一週要換一個世界觀 (M7)

- 生成式模型依上下文產生新內容，也可以執行分類、抽取等任務
- ★ 評估方式、失敗模式與風險型態因此不同
- 單一「準確率」不足以涵蓋正確性、依據、格式、安全與實用性

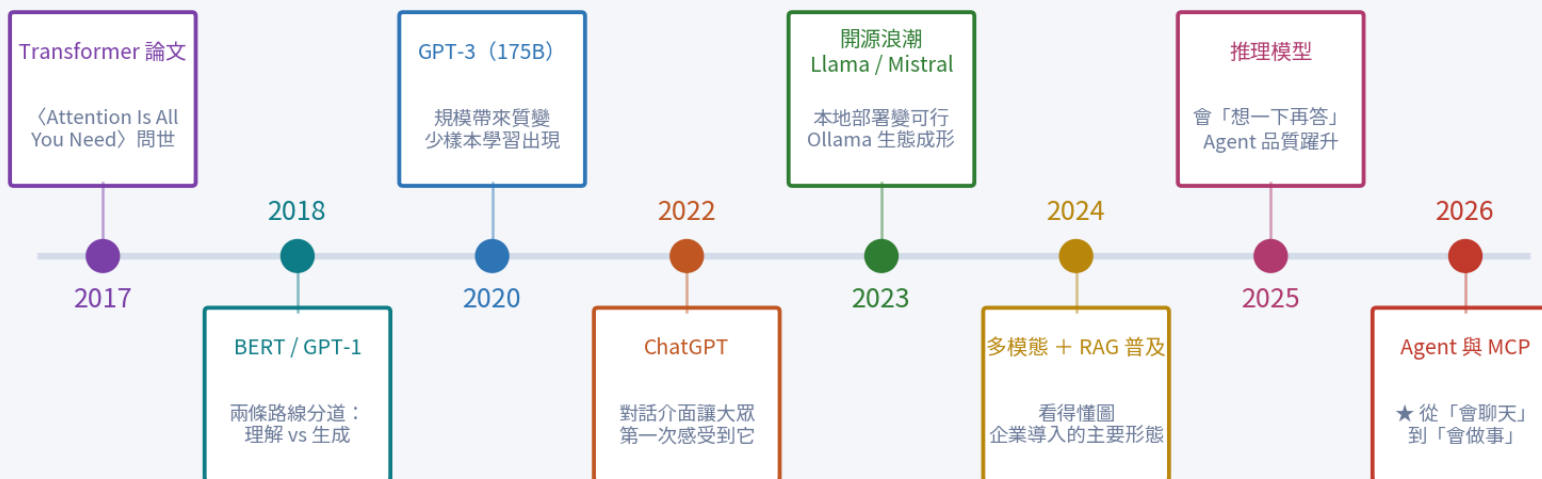
為什麼要先學原理再學工具 (M8)

- M8 在 Dify 裡每一個欄位——Top-K、Temperature、上下文長度、模型選擇
- 背後都是這一週的概念。不懂原理，你只能照抄別人的設定值
- ★ 出事的時候 (幻覺、成本爆掉、答非所問) 你會不知道從哪查起

本模組以原理為主，實作包含 AI Studio 生成式範例與 Ollama 本地部署。

技術演進：從一篇論文到會做事的 Agent

生成式 AI 的技術演進：從一篇論文到會做事的 Agent



- ★ 注意兩件事：① 從 2017 到 ChatGPT 隔了五年——技術突破與大眾感知之間有很長的落差。
② 2023 年之後每一年的主題都在往「能做事」移動，這正是本課程 M8 期末專題要做的方向。

2017 是關鍵架構轉折點

〈Attention Is All You Need〉提出 Transformer。生成式模型早已存在，但現代 LLM 大多沿著這條架構主線發展。

★ 注意那五年的落差

2017 技術突破 → 2022 ChatGPT 才讓大眾感受到。
技術成熟與應用普及之間，永遠有一段時間差。

2023 之後的主題一直在移動

開源本地化 → 多模態與 RAG → 推理 → Agent。
重點逐漸從「會聊天」移向「能連結資料、工具與流程」。

這正是課程的終點

M8 期末專題做的就是 2026 年這一格：一個會做事的 Agent。

三個轉折點，各自解決了什麼問題

轉折	之前的困境	解決了什麼	留下什麼新問題
2017 Transformer	RNN 依序處理，長距離依賴較難學；訓練不易沿序列平行化	同一層可直接比較不同位置；訓練更易平行化	計算量隨長度平方成長 → 上下文有上限
2022 ChatGPT	模型能力已具備，但一般人接觸不到、也不知道能拿來做什麼	對話介面大幅降低一般使用門檻	★ 大眾開始把它當成「知識來源」，而它其實不是
2024–26 RAG 與 Agent	模型知識過時、不知道你的資料、只會講不會做	RAG 提供外部依據；工具調用連結可執行動作	可靠性與可稽核性成為主要瓶頸——本課程的重心

★ 看得出一個規律：每次突破都會解決舊問題、同時製造新問題。目前這一輪的新問題，就是你們要處理的。

PART 01

生成式與判別式

Generative vs Discriminative

同樣叫做「模型」，但這兩類東西學的是完全不同的東西。
搞清楚這個差別，你就懂了一半。

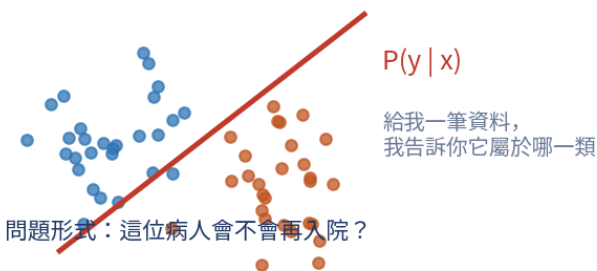
兩類模型的根本差別

生成式與判別式：你前六週學的，跟接下來要學的差在哪

判別式模型 Discriminative

M1-M3 學的全部都是這一類

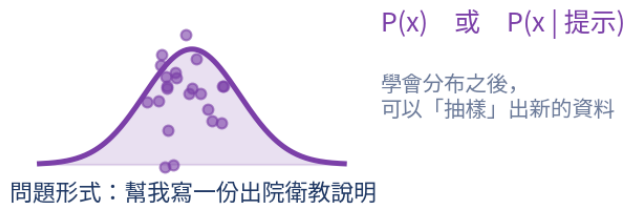
學的是「分界線」



生成式模型 Generative

M7-M8 要學的是這一類

學的是「資料長什麼樣」



★ 關鍵差別：判別式回答「是哪一類」，生成式產生「一個新的東西」。這也是為什麼生成式模型會「編」——它本來就是設計來產生新內容的。

判別式：學分界線

$P(y|x)$ 。給一筆資料，回答它屬於哪一類。
M2-2 的所有分類器都是這一類。

生成式：學資料分布

學習 $p(x)$ 或 $p(x,y)$ 等資料分布；語言模型則學
 $p(\text{token} | \text{context})$ ，再逐步取樣產生內容。

★ 為什麼生成式會「編」

它的任務就是依條件產生未見過的内容；
但「新」不代表「有事實依據」或「可以直接採用」。

評估方式也不同

分類常看 F1、AUROC 等；生成還要評估
事實與依據、任務成功、
安全、穩健性與人工可接受度，沒有單一
指標足夠。

具體對照：同一個醫療場景，兩類模型各自能做什麼

情境	判別式模型 (M1–M3)	生成式模型 (M7–M8)
再入院風險	輸入 30 個特徵 → 輸出「高風險 0.82」	以經驗證的監督式預測模型為主；LLM 可協助文字特徵抽取，但不取代風險模型
出院衛教說明	做不到。它只能選類別，不能寫文章	★ 可草擬個別化說明；內容需有核准依據並由專業人員覆核
病歷摘要	做不到	★ 可草擬 SOAP / ISBAR 摘要；必須核對遺漏、時間與主體
檢驗數值異常偵測	★ 用 Isolation Forest 或統計法，穩定可稽核	數值規則交給程式或統計模型；LLM 只負責說明已驗證結果
衛教問答	做不到 (除非做成固定 FAQ 比對)	★ 理解自然語言問題並回答
長照等級判定	可做成分類器，但需大量標註資料	資格判定應依官方規則與可測試流程；LLM 可協助檢索與說明，不做最終裁定

★ 判準不只看輸出形式：固定規則、風險預測與數值計算優先用可驗證方法；生成式模型適合草擬、轉換與互動。

六大應用領域：醫護場域的具體例子

生成式 AI 的六大應用領域：醫護場域的具體例子

文字生成 Text

- 衛教說明生成
- 病歷摘要
- 出院指導單
- 公文與報告

文字理解 Understanding

- 分類與分級
- 關鍵資訊抽取
- 情緒與滿意度
- 檢索與問答

影像 Vision

- 醫學影像輔助判讀
- 報告掃描檔 OCR
- 衛教圖示生成
- 手寫病歷辨識

語音 Audio

- 門診對話轉文字
- 衛教語音導覽
- 長者語音互動
- 會議紀錄

程式碼 Code

- 資料清理腳本
- 分析流程自動化
- ★ M10 Vibe Coding
- 儀表板產生

多步任務 Agent

- 個案評估流程
- 資料查核與通報
- ★ M8 期末專題
- 跨系統整合

★ 護理與健康科技學生的機會多半不在「做模型」，而在「知道哪個場景值得做、以及做出來能不能用」。

文字生成與理解

目前工具與範例最多、也較容易建立評測的兩塊。

本課程 M8 的專題多半落在這裡。

影像與語音

醫學影像判讀屬於高度管制領域；
但報告掃描 OCR、門診對話轉文字已相當實用。

程式碼

★ M10 的 Vibe Coding。對非資工背景的你，
這可能是投報率最高的一塊。

★ 多步任務 (Agent)

M8 期末專題的方向。Agent 需要額外處理權限、工具錯誤與追蹤紀錄。

護理與健康科技領域的六個可行切入點

切入點	做什麼	難度	資料取得
衛教內容個別化	依病人年齡、識能、疾病產生客製衛教說明	低	公開衛教素材
長照給付諮詢	依公開規則檢索申請條件與服務；正式資格仍依主管機關流程	中	★ 公開法規（本課程範例）
病歷摘要與交班	把長篇紀錄整理成 SOAP 或 ISBAR	中	需去識別化，取得較難
問卷與量表輔助	說明題意與彙整公開回覆；不可代替受試者作答	低	自建題庫
文獻查證助理	針對臨床問題檢索並摘要實證文獻	中	PubMed 等公開來源
照顧者支持	依核准衛教與資源回答一般問題；高風險問題轉介專業人員	低	★ 公開衛教與社會資源

★ 優先選公開、版本可追蹤的資料；期末專題不得使用真實病歷或個案資料。

四種生成式模型架構

四種生成式模型架構：各自擅長什麼



GAN

生成器與鑑別器對抗學習。
仍用於影像合成、資料增強與特定生成任務。

VAE

用機率式潛在空間編碼再解碼。
常見於表示學習、異常偵測與生成。

Diffusion

從雜訊逐步去噪產生樣本。
是影像生成的重要架構家族。

★ Transformer

以注意力為核心的架構家族，是現代 LLM 的主流基礎。
本課程接下來聚焦在這一個。

PART 02

Token：模型眼中的文字

Tokenization

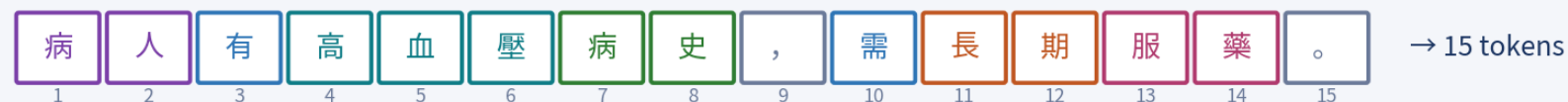
模型看不到「字」，它只看到編號。

這件事同時決定了你的帳單、上下文上限，以及它為什麼數不清字數。

Tokenization 實例

Tokenization：模型眼中的文字長這樣

句子：「病人有高血壓病史，需長期服藥。」



中文：大致「一個字 $\approx$ 1 個 token」

- 上面 15 個字 $\rightarrow$ 約 15 tokens
- 同樣意思的英文約 10 tokens
- ★ 中文比英文貴，同樣內容 token 數多約 5 成

英文：1 token $\approx$ 0.75 個單字 $\approx$ 4 個字元

- "hypertension" $\rightarrow$ 可能被切成 hyper + tension
- "病人" 這種常見詞可能合併成一個 token
- 切法由該模型的 tokenizer 決定，各家不同

為什麼你必須知道 token 這件事

- ① 計費以 token 計 —— 輸入與輸出分開計價，輸出通常比輸入貴 3-5 倍
- ② 上下文長度以 token 計 —— System Prompt + 知識庫段落 + 對話歷史都佔額度
- ③ 模型看不到「字」，只看到 token 編號 —— 這是它算不準字數、數不清字母的根本原因

中文的粗估

沒有固定比例：同一中文字可能獨立成 token，也可能與其他字合併。

成本與長度要用目標模型的 tokenizer 實測。

英文的粗估

英文常見粗估約 4 個字元 / token，但模型、語料與符號都會改變結果。

少見詞可能被切成多個子詞 token。

切法各家不同

每個模型有自己的 tokenizer。

同一句話在不同模型算出的 token 數不一樣。

★ 實測工具

使用模型官方 tokenizer 或計數工具實測；不同模型不可直接共用估算。

特別檢查繁體中文、醫療縮寫、數字與特殊符號。

算一次帳：一個衛教問答機器人的月成本

用真實的 token 結構估算，數字比你想的具體

成本估算 (M8 會實際設定這些參數)

【單次問答的 token 組成】

System Prompt (四段式，含限制與格式)	約 450 tokens
知識庫召回段落 (Top-K = 5，每段約 300 字)	約 1,500 tokens
對話歷史 (前三輪)	約 600 tokens
使用者本次提問	約 30 tokens

輸入合計	約 2,580 tokens
模型輸出 (四段式回答)	約 400 tokens

【成本估算】 (費率隨模型與時間變動，此處僅示範算法)

假設輸入 US\$0.15 / 1M tokens、輸出 US\$0.60 / 1M tokens

$$\begin{aligned}\text{單次} &= 2,580 \times 0.15/1\text{M} + 400 \times 0.60/1\text{M} \\ &= 0.000387 + 0.000240 = \text{US\$}0.000627 \quad (\text{約新臺幣 } 0.02 \text{ 元})\end{aligned}$$

每天 500 次問答 $\times$ 30 天 = 15,000 次

月成本 = 15,000 $\times$ 0.000627 $\approx$ US\$9.4 (約新臺幣 300 元)

【看出什麼】

- ① 本例召回段落佔輸入 58%；Top-K、切塊長度與對話歷史都會影響成本
- ② 輸入 / 輸出費率、快取與推理 token 依模型與供應商而異
- ③ Agent 輪數與工具回傳不固定，必須另外設預算、步數與逾時上限

隨堂檢核 1 | 生成式與 Token

Q
1

為什麼「模型準確率 92%」這句話用在 LLM 上幾乎沒有意義？

準確率需要有標準答案可比對。生成式模型產生的是開放式內容，同一個問題有無數個合理答案，也有無數個看似合理但錯誤的答案。要評估得看事實正確性、格式一致性、有沒有幫上忙——多半要人工或用另一個模型來評。

Q
2

同樣一句話，中文和英文哪個 token 數多？這對你有什麼影響？

沒有固定答案；必須用各模型 tokenizer 實測。影響包括成本、可容納內容與切塊設計。

Q
3

為什麼 LLM 常常數不清一句話有幾個字？

模型以 token 表示文字，token 與字元不是一對一，所以純生成不適合做精確字數統計；需要時交給程式計算。

Q
4

上面的成本估算裡，哪一項最值得優化？

先量測召回段落與對話歷史。調整 Top-K、切塊、rerank 與快取可能降成本，但必須同時用代表性問題檢查召回率與答案品質。

PART 03

Attention 與 Transformer

The Attention Mechanism

2017 年那篇論文的標題就是〈Attention Is All You Need〉。
整個生成式 AI 的爆發，起點就在這個機制。

Attention 在做什麼

Attention 在做什麼：一句話說完



模型讀到「他」的時候，要決定「他」指的是誰

Attention 的做法：對句子中每一個詞算一個權重，權重高的就是關聯強的。

上圖線條越粗＝權重越高。模型把注意力集中在「病人」上——它「知道」他指的是病人。

為什麼這件事是突破

- 舊做法 (RNN/LSTM)：一個字一個字往後讀，讀到「他」時前面的「病人」已經模糊了
- Attention：一次看全部，直接算出「他」跟每個詞的關聯強度——距離多遠都不衰減
- 代價：要算「每個詞對每個詞」的關聯，計算量隨長度平方成長 → 這就是上下文長度有上限的原因

問題

模型讀到「他」的時候，怎麼知道「他」指的是誰？

做法

以 Query、Key、Value 計算位置間的注意力權重。

權重反映此層的資訊混合，不等於可直接解讀的因果證據。

★ 突破在哪

RNN / LSTM 沿序列傳遞狀態；Self-Attention 在同一層直接比較多個位置，也讓訓練更容易平行化。生成下一個 token 仍是逐步進行。

代價

標準全注意力要比較所有位置，時間與記憶體約隨長度平方成長。

現代模型會用快取、稀疏或高效注意力，但長上下文仍有成本。

為什麼 Attention 解決了醫療文本的核心難題

醫療文本的特性

- ▶ 長距離指涉：「該病人」「上述症狀」「此藥物」常常指涉很前面的內容
- ▶ 巢狀修飾：「因高血壓合併糖尿病而長期服用之降血壓藥物」
- ▶ 縮寫與同義詞：HTN / 高血壓 / hypertension 指同一件事
- ▶ 否定與條件：「無胸痛」「若出現發燒則...」的語意要能正確捕捉

Attention 怎麼處理

- ▶ 長距離指涉 → 在 context window 內可直接比較位置，但仍受位置偏誤影響
- ▶ 巢狀修飾 → 多頭注意力可學不同關係，但每個 head 不一定可人工命名
- ▶ 縮寫同義 → 若訓練資料涵蓋，表示空間可能接近；醫療語料仍要驗證
- ▶ ★ 否定、時間、不確定性與主體都要用標註集測試

★ 醫療 NLP 應另外驗證否定偵測 (Negation Detection)、時間與不確定性；必要時搭配規則或專用模型。

三種 Transformer 架構

三種 Transformer 架構：為什麼 RAG 要用兩種不同的模型



Encoder-only (BERT)

以雙向注意力建立上下文表示，擅長理解與抽取。

★ 許多嵌入模型採 Encoder-only 或專用編碼架構。

Decoder-only (GPT)

使用 causal mask，只根據已見 token 預測下一個 token。

★ 多數對話式 LLM 採 Decoder-only。

Encoder-Decoder (T5)

先讀完再生成。擅長「轉換」：翻譯、摘要。

AI Studio 的 T5 範例屬於這類。

★ 為什麼 RAG 要兩種模型

RAG 通常用嵌入模型把文字變向量，再用生成模型產生答案。

兩者可來自不同架構、版本與供應商。

三種架構的實際用途對照

架構	代表模型	訓練方式	在你的專題裡的角色
Encoder-only	BERT、RoBERTa、 bge / text-embedding 系列	克漏字 (遮住一個詞讓它猜)	★ 常見用途：把段落與問題轉成可比較的向量
Decoder-only	GPT、Claude、Gemini、 Llama、Qwen、DeepSeek	預測下一個 token	★ 主力：生成回答、判斷分類、Agent 的大腦
Encoder-Decoder	T5、BART、mT5	把一段序列轉成另一段序列	翻譯、摘要等固定轉換任務；AI Studio 範例用 T5

★ 在 Dify 等 RAG 系統中，嵌入模型與生成 LLM 分開設定；換嵌入模型通常需要重新索引。

Transformer 不只 Attention：三個不能漏掉的元件

① 位置資訊 Positional Information

- Token embedding 本身不含順序
- 原始 Transformer 加入 positional encoding
- 現代模型也常用 RoPE 等位置表示

② 遮罩 Attention Masking

- Encoder 可同時利用左右文
- Decoder 用 causal mask 阻擋未來 token
- 架構相近，遮罩與訓練目標不同

③ 殘差、正規化與 MLP Residual / Norm / MLP

- Attention 只是 Transformer block 的一部分
- 前饋網路 (MLP) 逐位置轉換特徵
- 殘差連接與正規化讓深層訓練穩定

Embedding：語意的數學表示

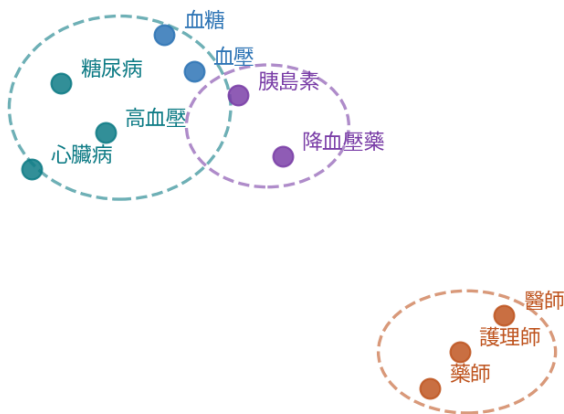
Embeddings & Semantic Space

文字沒辦法直接計算。要讓電腦判斷「兩句話意思像不像」，得先把文字變成有「距離」的數字。

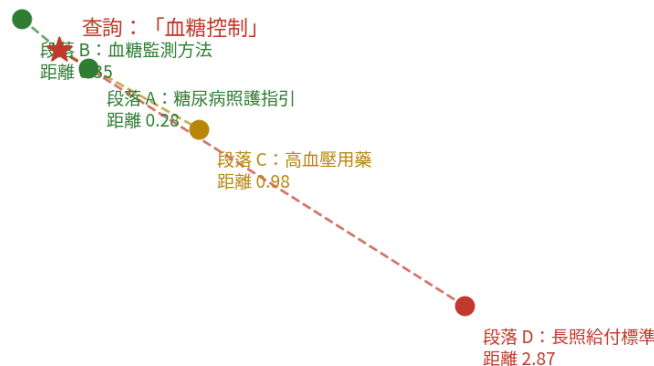
Embedding：把文字變成有距離的數字

Embedding：把文字變成有「距離」的數字

語意相近的詞，向量位置也相近



RAG 的召回：找出離查詢最近的段落



★ 這就是 M8 知識檢索節點的原理：把問題與所有段落都轉成向量，再挑距離最近的幾段餵給 LLM。

左圖 語意群聚

疾病名稱聚在一起、藥物聚在一起、醫療人員聚在一起。

這些群聚來自訓練目標與語料統計，不等於臨床分類或因果關係。

右圖 RAG 的召回

把問題也轉成向量，算出離它最近的段落。

★ 這是向量檢索的核心；實務還包含切塊、索引、混合檢索、rerank 與門檻。

為什麼是「語意」不是「關鍵字」

查詢「血糖控制」可能召回「糖尿病照護指引」，

即使沒有完全相同詞彙；效果取決於模型與語料。

維度

實務上是幾百到幾千維。圖上畫成二維只是為了看得懂。

實際算一次：餘弦相似度

RAG 用來衡量「像不像」的計算，本質上就這麼簡單

手算範例 (數值已驗算)

【簡化示範】把每個句子表示成 4 維向量 (實務是 768 或 1536 維)

查詢 Q	「病人的血糖偏高」	→ [0.8, 0.6, 0.1, 0.0]
段落 A	「糖尿病的血糖監測方式」	→ [0.7, 0.7, 0.1, 0.1]
段落 B	「長照給付額度標準」	→ [0.1, 0.0, 0.9, 0.4]

【餘弦相似度】 $\cos(Q,A) = (Q \cdot A) / (|Q| \times |A|)$

$Q \cdot A = 0.8 \times 0.7 + 0.6 \times 0.7 + 0.1 \times 0.1 + 0.0 \times 0.1 = 0.56 + 0.42 + 0.01 + 0 = 0.99$
 $|Q| = \sqrt{0.64 + 0.36 + 0.01 + 0} = \sqrt{1.01} = 1.005$
 $|A| = \sqrt{0.49 + 0.49 + 0.01 + 0.01} = \sqrt{1.00} = 1.000$
 $\cos(Q,A) = 0.99 / (1.005 \times 1.000) = 0.985 \quad \leftarrow \text{很相似 } \checkmark$

$Q \cdot B = 0.8 \times 0.1 + 0.6 \times 0.0 + 0.1 \times 0.9 + 0.0 \times 0.4 = 0.08 + 0 + 0.09 + 0 = 0.17$
 $|B| = \sqrt{0.01 + 0 + 0.81 + 0.16} = \sqrt{0.98} = 0.990$
 $\cos(Q,B) = 0.17 / (1.005 \times 0.990) = 0.171 \quad \leftarrow \text{不相似 } \times$

【對應到 RAG】若本示例暫設 Score 閾值 0.5：

$0.985 > 0.5 \rightarrow A$ 通過； $0.171 < 0.5 \rightarrow B$ 被過濾。

★ 真實門檻依嵌入模型、索引與語料調整；不同模型分數不可直接比較。

Embedding 的三個實務陷阱

① 中文的嵌入模型要選對

- 語言支援不只看模型名稱，要讀模型卡並用自家查詢測試
- 繁體中文、縮寫、表格與醫療術語可能表現不同
- ★ 換嵌入模型後通常要重新建立索引

② 嵌入模型換了，舊向量就不能用

- 不同嵌入模型通常位於不同向量空間，分數與維度不可混用
- 換模型要重新嵌入、重建索引並重新調整門檻

③ 語意相似 ≠ 答案正確

- 相似度只產生候選段落，不保證包含答案或支持最終陳述
- 需分開測 retrieval、groundedness 與 citation correctness
- ★ 「有引用」也要檢查引用是否真的支持該句

PART 05

預訓練、微調、推論

Pretraining / Fine-tuning / Inference

這三件事常被混為一談，但資料、硬體、時間與治理需求差異很大。
分清楚它們，才知道模型權重有沒有被改變。

三個階段的成本與角色

預訓練、微調、推論：三件常被混為一談的事



★ M8 的 Dify 只做第 ③ 階段。它不訓練也不微調 —— 這一句話決定了 Dify 能做與不能做什麼。

① 預訓練

前沿基礎模型可能使用兆級 token 與大型 GPU 叢集。

本課程不做從零預訓練。

② 微調

資料量與成本依模型、方法與任務而異；PEFT / LoRA 可降低資源需求。

有明確行為目標與評測集時才考慮。

③ 推論

★ 呼叫模型取得輸出就是推論；權重不變。

雲端常按 token / 請求計費，本地則消耗自有算力。

★ 對 M8 的意義

本課程在 Dify 主要做推論、RAG 與工具流程；

若要微調，通常在外部分完成後再接入模型服務。

模型適配的五種常見路徑（不是同一層次分類）

路徑	改變什麼	資料 / 資源	適用情境
持續預訓練 Continued Pretraining / DAPT	以領域文本繼續做語言模型訓練	大量未標註領域語料 + 訓練算力	補足術語與領域語體；不直接教任務答案
監督式微調 Supervised Fine-tuning, SFT	用輸入→理想輸出範例調整行為	高品質指令與答案；需獨立評測集	固定任務、格式、語氣與拒答行為
參數高效微調 PEFT / LoRA	只更新少量 adapter / 低秩參數	與 SFT 類似資料；較低顯存與版本成本	資源有限、要維護多個任務版本
偏好最佳化 RLHF / DPO	依人類偏好或 chosen / rejected 對調整	偏好排序、成對答案或回饋模型	對齊語氣、幫助性、安全與拒答偏好
資料策略 Synthetic Data / Distillation	用教師模型產生或篩選訓練資料	少量真實標註 + 合成資料 + 品質篩選	擴充稀缺案例；須防止錯誤被放大

★ DAPT、SFT、PEFT 與偏好最佳化分屬訓練階段、目標與參數策略；不要把所有「再訓練」都叫同一種微調。

要讓模型「懂我的領域」，有四種辦法

要讓模型「懂我的領域」，有四種辦法（由便宜到昂貴）

做法	時間	成本	知識更新	適合處理什麼
提示詞工程 Prompt	分鐘	幾乎為零	改一行字就好	行為與格式的調整
RAG 知識庫 Retrieval	小時	低（存向量庫）	換文件即可	★ 知識會變動的場景
微調 Fine-tuning	天～週	中～高	要重新訓練	風格、術語、格式的深度內化
重新預訓練 Pretraining	月	極高	不可能	不用考慮
★ 決策順序：先試提示詞 → 不夠加 RAG → 還不夠才考慮微調。多數人一開始就想微調，那總是錯的。				

由便宜到昂貴

先定義任務與評測集，再比較提示詞、RAG 與模型適配。

成本通常遞增，但方法可以組合，沒有固定單一路徑。

★ 決策順序

先做最簡單可測的基準；缺外部知識用 RAG，缺穩定行為再評估 SFT / PEFT。每一步都要用同一評測集比較。

知識會變動 → 選 RAG

法規、給付標準與指引會改版。

RAG 較容易更新、版本化與回傳來源，但仍要驗證引用是否支持答案。

風格要內化 → 才考慮微調

需要大量、穩定地遵循院內格式、語氣或任務行為，

且提示詞基準仍不足時，再評估 SFT / PEFT。

RAG 與微調：一張表講清楚差別

面向	RAG	微調
解決什麼	缺少可更新、可追溯的外部事實	模型行為、格式或任務表現不穩
知識存在哪	外部知識庫 (可隨時替換)	模型權重 / adapter ; 更新需新版本與再訓練
更新一份法規	替換文件，重新索引，幾分鐘	重新訓練，數小時 ~ 數天
可以標註來源嗎	★ 可以回傳召回段落；仍須檢查引用與陳述是否一致	無法由權重直接追溯到單一訓練來源
成本	需索引、檢索、rerank 與額外 context token	需訓練、評測、版本管理與回滾；PEFT 可降成本
醫療場域適用度	常適合變動知識；可更新與回傳來源，但不是自動可稽核	可用於格式與行為；高風險場景仍需獨立驗證與治理
兩者可以並用嗎	★ 可以，也常這麼做：微調負責格式，RAG 負責事實	

★ 醫療場域常先評估 RAG，因為知識可更新、可回傳來源；但可靠性仍取決於檢索、生成與引用驗證。

PART 06

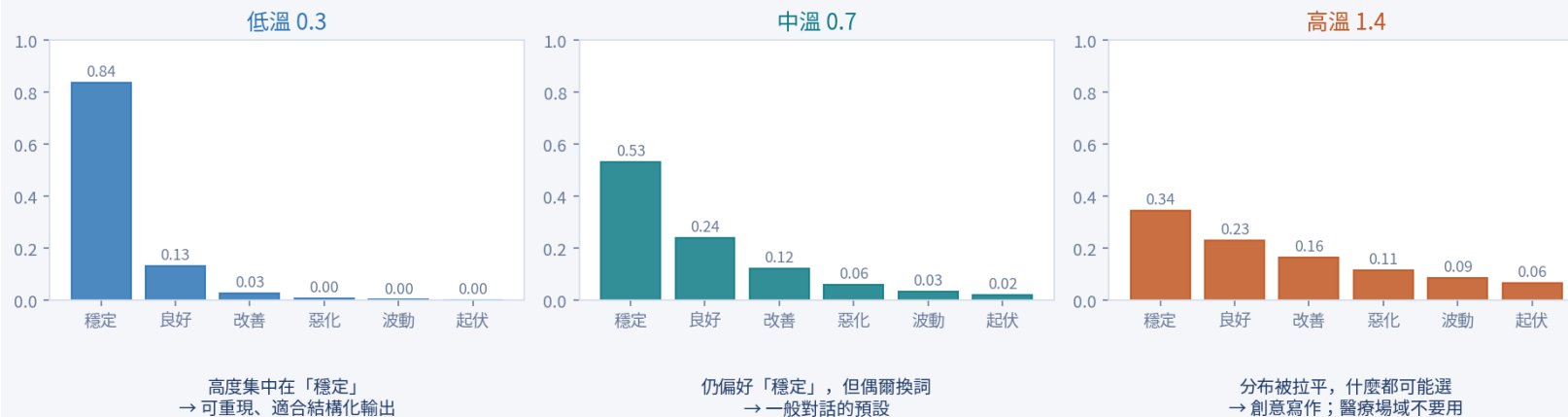
推論參數與幻覺

Inference Parameters & Hallucination

同一個模型、同一個問題，設定不同答案就不同。
以及那個所有人都在問的問題：它為什麼會胡說八道？

Temperature：控制隨機性

Temperature：同一句話「病情...」，下一個詞的機率分布怎麼變



★ 醫療應用一律用低溫（0-0.3）。Temperature 調低能減少「用詞飄移」，但擋不住「內容編造」——那要靠 RAG。

它在調什麼

下一個 token 的機率分布。溫度低 → 分布尖銳、幾乎總選最高機率的；
溫度高 → 分布拉平、什麼都可能被選中。

低溫 0-0.3

輸出變異通常較低。高風險任務應先追求穩定，再依模型與評測調整。

中溫 0.7

可增加變化；沒有跨模型通用的「最佳預設值」。

★ 重要的限制

Temperature 調低可減少取樣變異，但不能保證事實正確。
還要靠外部依據、工具驗證、拒答與人工覆核。

其他常見推論參數

參數	作用	設定原則	調錯會怎樣
Temperature	控制隨機性	依穩定性與任務評測調整	太高 → 用詞飄移、格式不穩
Top-p (核採樣)	只從累積機率前 p 的候選中挑	通常只調 Temperature 或 Top-p 之一	太高 → 冷僻詞被選中的機會上升
Max tokens	限制輸出長度	依格式估算，留 20% 餘裕	太小 → 答案被硬生生截斷
Frequency penalty	抑制重複用詞	保留術語一致性並實測	太高 → 該重複的術語被迫換詞，反而錯
Seed	提供最佳努力的重現；仍受平台與模型版本影響	測試時可固定，並記錄模型 / 版本 / 參數	固定 seed 也未必完全相同；要保存輸入、版本與輸出

★ 課堂練習：在同一模型與版本下改變 Temperature，多次執行並比較變異；不要把數值直接搬到別的模式。

幻覺是怎麼來的

幻覺是怎麼來的：從機率的角度看

LLM 實際在做的事（不是你以為的那件事）



① 訓練資料的空白處

模型沒學過的內容，它不會說「不知道」，而是產生一個「看起來合理」的答案

② 機率取樣的本質

每個字都是抽樣出來的。
抽到低機率但錯誤的字，整句就歪了

③ 沒有事實查核機制

模型內部沒有「這是真的嗎」這個步驟。
它只有「這順不順」

★ 所以「幻覺」不是 bug，是這個機制的必然產物。

你不可能用提示詞「關掉」它 —— 只能用 RAG 提供事實、程式碼接管數值、用人工覆核把關。這正是 M8 整個模組在做的事。

它在做的事

依上下文估計下一個 token；訓練目標本身不等於外部事實查核。

三個來源

① 資料與知識缺口 ② 不完整 / 衝突的 context ③ 生成與查核目標分離

★ 幻覺是需要持續管理的系統風險

無法假設用一條提示詞就能消除；風險會隨任務、模型與 context 改變。

能做的三件事

用 RAG / 工具提供依據、程式接管數值、拒答與人工覆核把關；
並以測試集持續監測，而不是只看示範成功。

醫療場域的幻覺特別危險：四個常見失敗樣態

為什麼比其他領域嚴重

- ▶ 錯誤資訊可能與正確資訊同樣流暢，使用者難以判斷
- ▶ 劑量、頻次與禁忌症等錯一個數字就可能造成傷害
- ▶ ★ 焦慮情境與權威語氣會放大自動化偏誤
- ▶ 模型可能過度自信，也可能用含糊語氣掩蓋錯誤

三個常見樣態

- ▶ ① 編造數值：說出一個沒人量過的檢驗值或給付金額
- ▶ ② 拼接不同來源：把 A 疾病的處置說成 B 疾病的
- ▶ ③ 過時資訊：引用已經被修訂的舊版指引或給付標準
- ▶ ④ 過度推論：從「有這些症狀」跳到「你可能得了某某病」

★ 免責聲明不是安全控制。還要限制用途、引用核准來源、設定拒答與轉介、專業覆核、監測與回滾。

隨堂檢核 2 | 三階段與幻覺

Q
1

你的專題要讓 AI 熟悉最新版的長照給付標準。該微調還是用 RAG？

RAG。給付標準每年會修訂，微調把知識鎖進權重，更新只能重訓；RAG 換一份文件、重新索引就好，而且可以標註來源段落——這在稽核上是硬需求。

Q
2

Temperature 設成 0，模型就不會產生幻覺了嗎？

不會。Temperature 只影響「從候選中怎麼挑」，不影響「候選本身對不對」。設成 0 只是讓它每次都挑同一個最高機率的答案——如果那個答案本來就是錯的，它會穩定地錯下去。

Q
3

為什麼 RAG 可以標註來源，微調不行？

RAG 可回傳檢索段落與來源；但仍要驗證引用是否真的支持回答。微調權重則無法直接追溯到單一訓練樣本。

Q
4

一個病人問「我這個藥可以配酒嗎」，你的衛教機器人該怎麼設計？

這已超出一般衛教範圍。System Prompt 應限制不提供個別用藥建議，偵測後轉介藥師或醫師並提供聯絡方式。即使有 RAG，也不能把召回資料當成可直接給病人的處方建議。

PART 07

模型規模、能力與選型

Model Scale & Capabilities

參數量、上下文長度、多模態——這些數字到底代表什麼，
以及哪些其實已經不重要了。

衡量一個模型的五個面向

面向	是什麼	實務上的意義
參數量 Parameters	模型內部權重的數量，7B、70B、數千億不等	影響記憶體與運算，但能力還取決於架構、資料、訓練與量化
訓練資料量 Training Tokens	預訓練看過多少 token，兆級起跳	需連同資料品質、配比與訓練方法看；token 數多不等於資料好
上下文長度 Context Window	單次能處理多少 token	★ 標稱上限不等於有效使用長度；長文任務要測位置偏誤、召回與成本
多模態 Multimodal	能不能處理影像、音訊、影片	先確認輸入模態與用途；能讀影像不代表具備醫療判讀效能
推理能力 Reasoning	能不能做多步推理而不只是接話	影響多步任務表現；仍需工具驗證、權限限制與失敗回復

※ 具體模型與數字變動極快（本頁整理於 2026 年 7 月）。上課時請以各家官方文件為準——這也是要教你「看哪些面向」而不是「背哪個模型最強」的原因。

選模型時真正該問的四個問題（上）

① 資料能不能出機構？

- 先釐清資料分類、授權、契約、保存、資料區域與存取控制
- ★ 課程禁止把真實病歷或個案資料送入任何外部 AI 服務
- 去識別化後仍要做重識別風險與機構合規評估

② 中文（尤其繁體與醫療用語）表現如何？

- 國際排行榜不等於繁中醫療表現
- 先用 20 題做 pilot，再擴充成涵蓋常見、罕見與拒答案例的代表性評測集
- ★ 固定版本、提示詞與評分規準，才能公平比較

這兩題決定「哪些模型可以進入候選」。過不了這兩關，能力再強也不能用。

選模型時真正該問的四個問題（下）

③ 成本結構撐不撐得住預期用量？

- 用前面教的方法估算：輸入 token × 費率 + 輸出 token × 費率 × 次數
- 注意 Agent 的輪數不固定，成本上限要另外設
- 免費額度用完之後的單價才是你真正要算的數字

④ 這個選擇好不好換？

- 換模型看似簡單，但提示詞、工具格式、tokenizer 與安全行為可能都要重驗
- ★ 避免把核心流程綁死在單一供應商的獨有功能
- 版本化 Prompt、評測集與模型設定，才能安全切換與回滾

★ 注意這四個問題裡，沒有一個是「哪個模型在排行榜上最強」。

推理模型：2025 年之後的主要進展

推理模型：2025 年之後的主要進展

問題：「78 歲個案，中風後右側偏癱，無法自行如廁與沐浴，配偶 75 歲。CMS 大約第幾級？」

一般模型：直接接話

「第 5 級。」

- 答案來得很快（1 秒內）
- 成本低（只產出幾個 token）

- ★ 但你看不到它怎麼想的
- ★ 對了也不知道為什麼對，錯了也查不出哪裡錯

推理模型：先想再答

〔思考〕 先看 ADL 項目：如廁、沐浴皆需協助 → 2 項
再看 IADL 與照顧者能力 → 配偶高齡，支持有限
對照量表分級門檻 → 落在第 5-6 級之間
偏癱屬中重度失能 → 取較高者
〔回答〕 約第 6 級，建議由照管專員實地評估確認

差在哪、以及什麼時候值得用

- 推理模型會先產生一段「思考過程」再給答案——這段思考也要算 token，所以成本高、速度慢
- 多步推理、需要對照規則、需要權衡的任務，品質提升明顯；單純問答則沒必要
- ★ 對醫療場域的價值：思考過程本身可以被檢視。專業人員能看出它哪一步推錯了，這比「一個答案」有用得多

- ★ 但要注意：思考過程「看起來合理」不代表它真的是這樣算的，也不保證結論正確。它仍然是生成出來的文字。

一般模型

直接產生回答，通常延遲與成本較低；仍可能做一定程度的多步推理。

推理模型

在回答前使用更多推論計算；可能回傳摘要或中間結果，但不等於公開真實內部計算。

★ 對醫療的價值

可能提升多步任務表現；專業人員仍應檢查輸入、依據、計算與最終結果，不可只審閱生成的理由。

★ 但別過度信任

可見解釋可能不完整或不忠實，也可能受提示偏誤影響。
把它當除錯線索，不是臨床稽核軌跡。

可見推理 ≠ 忠實內部思路：驗證結果、證據與穩健性

① 驗證結果 Outcome Correctness

- 用有標準答案或明確 rubric 的測試集
- 檢查最終結論與錯誤類型
- 不用「理由看起來合理」代替正確率

② 驗證依據 Evidence Traceability

- 保存檢索段落、工具輸入輸出與計算紀錄
- 逐句檢查引用是否支持陳述
- 高風險數值交由程式或權威資料源驗證

③ 驗證穩健性 Robustness

- 改寫問題、改變順序並加入干擾資訊
- 測罕見案例、拒答案例與邊界條件
- 高風險用途由具資格人員做最終判斷

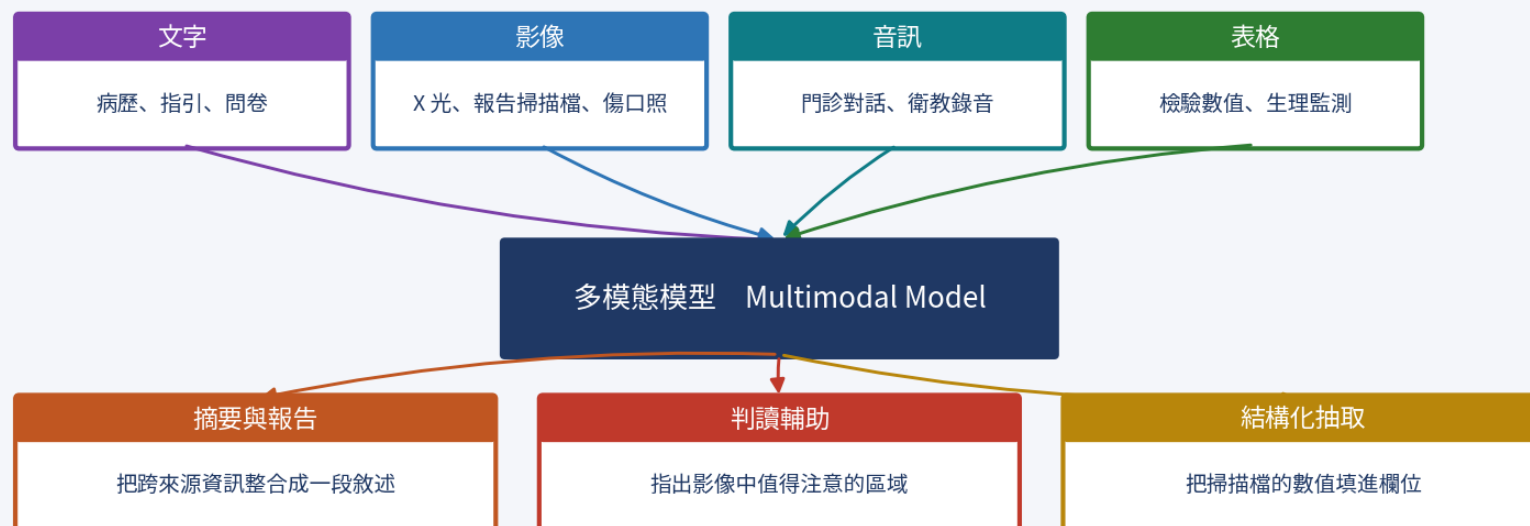
什麼任務值得用推理模型

任務型態	值得嗎	為什麼
查詢事實 (今天門診時間 ?)	✗ 不值得	單步查詢，推理沒有用武之地，只是白花錢
格式轉換 (散文改 SOAP)	✗ 不值得	規則明確，一般模型就做得好
多步判斷 (依量表判定等級)	★ 值得	要對照多個條件；應稽核規則、證據與結果，而非只看 CoT
拆解模糊任務 (Agent 規劃)	★ 值得	Agent 規劃可能受益，但仍需步數、權限與工具結果驗證
寫程式與除錯	★ 值得	M10 的 Vibe Coding 明顯受益
情緒支持與陪伴對話	✗ 不值得	通常不需額外推理成本；重點是同理、界線與安全轉介

★ 推理成本與延遲依模型、推理額度與任務而異。用固定評測集比較效益，不要預設最強模型就是最佳選擇。

多模態：不只讀文字

多模態：不只讀文字



★ 醫學影像判讀屬高度管制領域（醫材法規），課堂專題請避開「診斷」類應用。
相對安全且實用的方向：報告掃描檔 OCR、門診對話轉文字、衛教圖示生成。

輸入端

部分多模態模型可共同處理文字、影像、音訊或結構化輸入；支援範圍依模型而異。

醫護場域的實際用途

報告掃描檔 OCR、門診對話轉文字、傷口照片描述、
把掃描檔的數值填進結構化欄位。

★ 要避開的方向

醫學影像判讀可能涉及醫療器材、臨床驗證與專業責任。
課堂專題避開「診斷」類應用。

相對安全的切入點

OCR、語音轉文字與衛教圖示草擬相對適合起步；
仍要驗證錯字、遺漏、隱私與可近用性。

PART 08

模型跑在哪裡

Deployment: Cloud vs Local

對醫療與長照場域來說，這一節可能是整個模組最重要的部分。
因為它決定的不是效能，是資料會不會離開你的機構。

三種部署方式的取捨

模型跑在哪裡：三種部署方式與資料落地的取捨

雲端 API		本地部署		私有雲／地端企業版	
OpenAI • Anthropic • Google		Ollama + 開源權重模型		Azure OpenAI • 院內 GPU 叢集	
能力	最強	能力	中等 (8B-70B)	能力	強
成本	按 token 計費	成本	電費 + 一次性硬體	成本	高 (授權+維運)
資料	★ 送出機構外	資料	★ 完全不出機房	資料	合約內不外流
硬體	不用	硬體	16GB RAM 起, 有 GPU 更好	硬體	機房級
上手	最快	上手	中	上手	慢, 要採購
一般應用、課堂練習		★ 病歷等敏感資料		醫院正式導入	

★ 課程鐵則：真實病歷、個案紀錄、諮商記錄一律不得送進雲端 API。要練習就用去識別化或公開資料集。

雲端 API

通常能力與維運便利性高。

★ 資料會傳到外部服務；須查契約、保存、訓練使用、資料區域與存取控制。

本地部署 (Ollama)

資料可留在自有環境，但模型能力與維運由自己承擔。

記憶體需求依模型大小、量化、context 與是否用 GPU 而異。

私有雲 / 地端企業版

可談資料區域、隔離、稽核與服務承諾；仍需驗證契約與技術控制，不能把「私有」視為零風險。

★ 課程鐵則

課程不得把真實病歷、個案或諮商紀錄送入任何外部 AI 服務。

練習使用公開或虛構資料；去識別化仍需風險評估。

實作：用 Ollama 在自己電腦上跑一個 LLM

十分鐘可完成。這是本模組唯一必做的環境建置

Ollama 本地部署 (原講義 pp.60)

① 安裝：到 <https://ollama.com> 下載對應作業系統的安裝檔
Windows / macOS / Linux 都有；安裝後 ollama 指令即可用

② 下載並執行一個模型 (第一次會下載權重檔，數 GB)
ollama run qwen3:4b # 多語言 / 中文，約 2.5 GB
ollama run llama3.2:3b # 輕量文字模型，約 2.0 GB
ollama run gemma3:4b # 多語言 + 影像，約 3.3 GB

③ 直接對話 (在終端機裡)
>>> 請用三點說明高血壓的飲食注意事項

④ 用 API 呼叫 (Dify 本地部署時就是接這個)
curl http://localhost:11434/api/generate -d '{
 "model": "qwen3:4b",
 "prompt": "請用三點說明高血壓的飲食注意事項",
 "stream": false
}

⑤ 常用指令
ollama list # 看已下載的模型
ollama rm gemma3:4b # 刪除模型釋放空間
ollama ps # 看目前載入記憶體中的模型

硬體需求取決於模型檔、量化與 context；context 越長，記憶體需求越高
用 ollama ps 檢查 CPU/GPU offload 與實際配置

本地模型的能力落差：誠實的說明

小型本地模型常見優勢

- ▶ 格式轉換：把散文改寫成 SOAP 或條列
- ▶ 摘要既有文本：內容都在輸入裡，不需要它「知道」什麼
- ▶ 分類與抽取：判斷情緒、抽出關鍵欄位
- ▶ ★ 搭配 RAG 時表現會明顯提升——事實由知識庫提供

常見限制（必須實測）

- ▶ 廣泛知識、多步推理與長文一致性可能弱於大型雲端模型
- ▶ 繁中與醫療術語表現依模型與版本而異
- ▶ context、量化與硬體會影響速度與品質
- ▶ ★ 用自家代表性題目測試，不用參數量猜能力

★ 本地前處理 + 受控雲端推理是一種方案，但必須先做去識別化驗證、資料流向審查與最小必要傳輸。

PART 09

Hugging Face 與課堂實作

Model Hub & Hands-on

模型從哪裡來？以及在 AI Studio 裡實際跑一次生成式模型。

Hugging Face：模型的集散地

它是什麼

- 大量模型、資料集與 Spaces 的公開平台；數量持續變動
- 可下載權重、查看模型卡與評測、試用 Demo
- ★ 找模型只是起點，來源與檔案安全也要檢查

怎麼挑一個模型

- 看任務、語言、base model、模型卡與評測
- 檢查 intended use、限制、授權、資料與版本
- ★ 優先選可信組織與 safetensors；載入 pickle / 自訂程式碼前做安全審查

量化版本 (GGUF) 是什麼

- 降低權重精度可縮小檔案與記憶體需求
- 品質、速度與硬體效益依模型、量化方法與任務而異
- GGUF 是本地推論常見格式；仍要查看模型來源、授權與量化資訊

原講義：GAI 1. Introduction to Generative AI.pptx pp.57–59 (TheBloke / QuantFactory 模型庫、llama.cpp)

課堂實作 | AI Studio 的生成式模型範例

★ 對應 AI Studio Repository / Data Sciences Course Materials / Generative Model

開啟檔案

Generative Model/

- 1 Generating Text with GPT2.rmp
- 2 Sentiment analysis .rmp
- Basic LLMs.rmp
- Generative Models -T5 Base.rmp
- Finetune Text2Text Generation.rmp
- RAG - restaurant suggestion.rmp
- RAG-restaurant new data.csv

預期結果

- 溫度越高，同一提示詞的輸出差異越大
- T5 適合轉換任務，GPT2 適合續寫
- RAG 範例能引用到 csv 裡的實際資料
- 無 RAG 時模型會「編」出不存在的餐廳

操作步驟

- 1 開啟「Basic LLMs.rmp」，觀察它接的是哪個 Hugging Face 模型
- 2 執行「1 Generating Text with GPT2」，改變 temperature 各跑三次
- 3 比較同一個提示詞在不同溫度下的輸出差異
- 4 開啟「Generative Models -T5 Base」，注意這是 Encoder-Decoder 架構
- 5 ★ 執行「RAG - restaurant suggestion」——這是 M8 的預告
- 6 在 RAG 範例中把 new data.csv 換成自己的資料，觀察召回結果變化
- 7 記錄：同一個問題，有無 RAG 的回答差在哪

本模組的 AI Studio 範例對照

範例檔	示範什麼	對應本講義
Basic LLMs.rmp	接 Hugging Face 模型、基本推論	PART 7、PART 9
1 Generating Text with GPT2.rmp	Decoder-only 的續寫行為、temperature 效果	PART 3、PART 6
2 Sentiment analysis .rmp	用預訓練模型做分類 (Encoder-only 的典型用法)	PART 3
Generative Models -T5 Base.rmp	Encoder-Decoder 的序列轉換	PART 3
Finetune Text2Text Generation.rmp	微調實際長什麼樣 (觀摩，不要求完成)	PART 5
RAG - restaurant suggestion.rmp	★ RAG 的最小可運行範例，M8 的預告	PART 4、銜接 M8

PART 11

提示詞基礎

Prompting Fundamentals

M8 會把提示詞系統化成四段式結構。

這一節先建立最基本的直覺：為什麼「怎麼問」會差這麼多。

提示詞的四個層次

提示詞的四個層次：同一個問題，四種問法

層次 1 直接提問 「什麼是高血壓？」

得到什麼：一段通用百科式說明。長度不定、格式不定、深淺不定。

問題：每次問結果都不一樣，無法用在系統裡。

層次 2 加上角色與對象 「你是衛教護理師，請向 70 歲長者說明高血壓」

得到什麼：語氣與用詞明顯調整，開始像個人在講話。

進步：可控性提高。但格式仍不穩定。

層次 3 加上結構與限制 「…分三點說明，每點 50 字內，不提藥物名稱」

得到什麼：格式穩定、長度可預期、危險內容被擋掉。

進步：★ 可以放進系統了。這是可用與不可用的分界線。

層次 4 加上範例與替代動作 「…格式如下範例；不確定時回答『建議洽詢醫師』」

得到什麼：格式高度一致，且知識邊界外有明確行為。

★ 這就是 M8 的四段式 System Prompt。

★ 多數人停在層次 1-2，然後抱怨「AI 不好用」。差別不在模型，在你給了多少約束。

層次 1 直接提問

得到一段通用說明。長度、格式、深淺都不受控。

層次 2 角色與對象

語氣與用詞開始調整，像個人在講話了。

★ 層次 3 結構與限制

格式穩定、長度可預期、危險內容被擋掉。
這是「可以放進系統」與「不行」的分界線。

層次 4 範例與替代動作

格式高度一致，知識邊界外有明確行為。
這就是 M8 的四段式 System Prompt。

同一個需求，四種問法的實際輸出對照

需求：向 70 歲高血壓長者說明飲食注意事項

實際輸出對照（節錄）

【層次 1】「什麼是高血壓的飲食注意事項？」

→ 高血壓患者應注意鈉的攝取，建議每日鈉攝取量低於 2300 毫克…（續 400 字）

問題：太長、有數字（來源不明）、用詞偏學術、每次結果都不一樣

【層次 2】「你是衛教護理師，請向 70 歲的高血壓長者說明飲食注意事項」

→ 阿公阿嬤您好，我來跟您說說吃東西要注意的地方…（續 300 字）

進步：語氣親切、用詞淺白 | 仍有問題：長度不定、沒有結構

【層次 3】+ 「分三點，每點 50 字以內，不提任何藥物名稱」

→ 1. 少放鹽：煮菜時鹽巴減半，可以用蔥薑蒜提味。

2. 少吃醃漬品：醬瓜、豆腐乳、香腸這類要盡量避免。

3. 多吃蔬果：每餐半碗青菜，水果一天兩份。

★ 進步：格式穩定、長度可控、危險內容被擋掉 → 可以放進系統了

【層次 4】+ 「格式如上範例；若問題超出飲食範圍，回答『這部分建議詢問您的主治醫師』；結尾固定加上免責聲明」

→ 格式與層次 3 相同，但問到「我可以停藥嗎」時會正確拒答，且每則結尾都有「本說明僅供參考，請依醫囑調整」

★ 進步：知識邊界外有明確行為 → 這才是可以交付的東西

三個立刻可用的提示詞原則

① 給替代動作，不要只給禁令

- ✗ 只寫「不要編造」缺少可執行的替代行為
- ✓ 「來源不足時回答查無資料，並轉介○○」
- ★ 拒答與轉介是重要控制，仍須搭配檢索與輸出驗證

② 範例比規則有效

- 規則說明邊界，範例示範實際輸入與輸出；兩者互補
- 範例會影響結構、語氣與詳盡程度，也可能帶入偏誤
- 醫療場域可用核准的 SOAP / ISBAR 範例並測試例外

③ 把「要什麼」寫清楚，比把「不要什麼」寫清楚更重要

- 明確指定任務、對象、來源、輸出格式與失敗時行為
- 模糊要求會增加輸出變異，但錯誤也可能來自資料或模型能力
- ★ 好提示詞像可測試的工作規格，不是保證正確的咒語

M8 W13 會把這些原則組織成可重複使用的四段式結構：角色 → 步驟 → 限制 → 格式。

上下文工程：不只是把 Prompt 寫好

上下文工程：不只是把 Prompt 寫好

模型單次能看到的內容（Context Window）就這麼大。裡面放什麼、放多少，是你要設計的。



↑ 一個典型 RAG 應用的上下文組成（總計約 5,000 tokens）

★ 召回段落佔最大宗

Top-K 從 5 調到 10，成本幾乎翻倍，品質未必變好

對話歷史會無限成長

超過上限就會被截斷——通常是最早的內容先被丟掉

「中間迷失」現象

放在上下文正中間的資訊，模型最容易忽略

★ 重要的東西放頭尾

System Prompt 放最前、使用者提問放最後，這是實務慣例

★ 業界近年把這件事叫「上下文工程」（Context Engineering）——比「提示詞工程」更貼近實務要解的問題。

上下文是有限資源

System Prompt、召回段落、對話歷史、工具結果都在搶同一個空間。

★ 召回段落佔最大宗

Top-K 增加通常會帶入更多 token；成本、延遲與品質需一起量測。

★ 「中間迷失」現象

長上下文模型可能出現位置偏誤（Lost in the Middle）；
先檢索、排序與壓縮，不要只靠把重要內容移到頭尾。

為什麼這個詞更貼切

「提示詞工程」聽起來像在雕字句；
實務上真正要解的是「有限 context 如何分配、排序、驗證與監測」。

上下文管理的四個實務手法（M8 會實際設定）

① 對話歷史要有策略地截斷

- 最簡單：只保留最近 N 輪，但早期的重要資訊會消失
- 較好：最近 N 輪 + 一段摘要壓縮更早的內容
- ★ 醫療注意：過敏、禁忌等關鍵資訊不能被截掉

② 召回段落要精不要多

- 可先召回較廣候選，再 rerank 後送入較少且高相關段落
- 候選數、保留數與門檻要用驗證問題調整，沒有固定 20→5

③ 把可以離線做的事移出上下文

- 查表、算術與格式驗證交給確定性工具，不讓 LLM 猜數字
- ★ 模型負責語言與規劃；程式負責可精確驗證的計算與規則

④ 監測實際用量，不要憑感覺

- Dify 的 Tracing 會顯示每次實際送出的 token 數與組成
- 上線後定期看：context 有沒有悄悄變胖、成本有沒有偏離預估

PART 12

界線與銜接

Ethics & Bridge to M8

技術能做到的事，不等於你應該做的事。
尤其在醫療與長照場域。

生成式 AI 在醫療場域的四條紅線

紅線	為什麼	課程要求
不得提供診斷	可能跨越系統核准用途、醫療器材規範與專業責任；不屬課堂範圍	限制用途、測試拒答與轉介，並由專業人員確認流程
不得提供處方與劑量	可能造成直接傷害；需權威資料、劑量檢核與具資格人員最終確認	課程不提供個別藥名與劑量建議；導向藥師或醫師
不得使用真實個案資料	涉及個資、契約、資安與研究倫理；資料流向必須事前核准	★ 課程只用公開或虛構資料；真實資料須另依 IRB、契約與院內政策
輸出必須標示 AI 生成與免責	透明揭露有助使用者理解用途、限制與責任歸屬	標示 AI 角色與限制；免責文字不能取代驗證、覆核與監測

★ 這四條在期末專題的評分中屬於「不符即扣分」項目，M9 會再深入討論偏誤與可解釋性。

還有兩個容易忽略的問題

① 偏誤會被放大，而且不易察覺

- 部署族群、語言與流程若不同於開發情境，效能可能下降
- 少數族群、罕見疾病與非典型表現需單獨測試
- ★ 分群報告錯誤與拒答率，不只看整體平均
- M9 會用實際案例深入討論

② 使用者會過度信任

- 流暢、有條理、專業語氣會提高主觀可信度
- 但流暢度不能證明正確性或來源支持
- ★ 用可核對引用、不確定性、限制說明與人工覆核降低過度信任

生成式 AI 上線前：用五個面向建立評測集

評估面向	怎麼測	通過條件 / 治理動作
事實與依據 Groundedness	代表性問題 + 權威答案；逐句檢查引用是否支持	符合預定正確率；無依據時拒答；引用可開啟並支持該句
任務成功 Task Success	依任務 rubric 評分：完整性、格式、遺漏與人工可接受度	達到 go / no-go 門檻；分析失敗案例，不只報平均
安全與拒答 Safety / Refusal	測危險、越權、知識庫外與提示詞注入	危險請求正確拒答與轉介；合法任務不過度拒答
穩健性與公平性 Robustness / Equity	改寫、順序、干擾與罕見案例；分群檢查錯誤率	版本更新後回歸測試；高風險族群由領域專家審查
隱私與維運 Privacy / Operations	測資料外洩、日誌、權限、版本漂移、監測與回滾	指定人類負責人；事件通報、停用與回復流程可執行

★ 先寫評測集與通過條件，再選模型與提示詞；上線後持續監測，不能只在 Demo 成功一次。

查證資料與延伸閱讀 (Selected References)

核心原理

[Attention Is All You Need](#) — Transformer / 位置編碼

[Hugging Face Tokenization](#) — BPE / WordPiece / SentencePiece

[Hugging Face Causal LM](#) — next-token prediction

[Lewis et al., NeurIPS 2020](#) — RAG

可靠性與評估

[NIST AI 600-1](#) — Generative AI Risk Profile

[Liu et al., TACL 2024](#) — Lost in the Middle

[Hsieh et al., RULER 2024](#) — effective context

[Anthropic 2025](#) — CoT faithfulness

部署與醫療治理

[Ollama Context Length](#) — 記憶體與 context

[Hugging Face Model Cards](#) / [Security](#) — 來源與安全

[衛福部 2026 生成式 AI 指引](#) — 醫療治理

[WHO LMM Guidance](#) — health AI ethics

★ 本頁採官方文件與原始研究；來源名稱可直接點選開啟。

M7 → M8：原理如何變成操作

M7 → M8：原理如何變成操作

M7（本週） 原理層

- Token 與 Tokenization
- Attention 與 Transformer
- Embedding 與語意空間
- 預訓練／微調／推論
- Temperature 等參數
- 幻覺的成因
- 本地 vs 雲端部署

M8（下週起） 應用層

- 估算 Token 成本與上下文上限
- 理解為什麼上下文有限
- 知識檢索節點的原理
- Dify 只做推論，不訓練
- 模型參數怎麼設
- 為什麼要 RAG 與程式碼節點
- 敏感資料選 Ollama

★ M7 的每一個概念，在 M8 都會變成一個你要設定的欄位或要做的判斷。
這一週看起來很理論，但它決定了下三週你會不會「知其所以然」。

左邊是這一週

看起來很理論的七個概念。

右邊是下三週

每一個概念在 Dify 裡都會變成一個你要設定的欄位，
或一個你要做的判斷。

★ 為什麼順序是這樣

先學工具再學原理，你會變成「照抄設定值的人」；
先學原理，你才知道為什麼要那樣設。

下週開始

M8 W13：Dify 平台入門與提示詞工程。
記得先把 Ollama 裝好。

Take-Home Assignment | M7 生成式 AI 基礎

W12 上週前繳交，上週期成績 50%

作業說明

【任務一】建立你自己的模型評測基準（40%）

1. 針對你打算做的期末專題領域，寫出 10 個具代表性的問題（例：長照給付、衛教問答、症狀分級—不要用真實個案）
2. 選 2 個模型：一個雲端（免費額度即可）、一個本地（Ollama）
3. 同樣的 10 個問題各問一次，記錄答案
4. 逐題標註：事實正確 ✓ / 部分正確 △ / 錯誤或編造 ✗
5. 統計兩個模型的分布，寫 200 字比較

【任務二】幻覺觀察（30%）

1. 設計 3 個「知識庫外」的問題（模型應該不知道答案的）
例：貴院某科的門診時間、某個虛構的給付項目
2. 觀察模型的反應：它會說不知道，還是會編？
3. 同一個問題問 5 次，記錄答案是否一致
4. 寫 150 字說明你觀察到的幻覺樣態

【任務三】Ollama 本地部署（30%）

1. 安裝 Ollama 並下載至少一個模型
2. 截圖：ollama list 的輸出 + 一次成功的對話
3. 寫 100 字：本地模型與雲端模型，你感受到的差異

【繳交】PDF 一份，含截圖與表格。檔名：M7_學號_姓名.pdf

【提醒】★ 不得使用任何真實病歷或個案資料，違者本次作業不計分。

本模組重點回顧

- 1 生成式模型依條件機率產生內容；新內容不等於有事實依據
- 2 Tokenizer 影響成本與 context；精確字數等計算交給程式
- 3 標準 self-attention 可直接比較位置，但長文仍有二次成本與位置偏誤
- 4 RAG 用 Encoder-only 做嵌入、Decoder-only 做生成——這是兩件事
- 5 ★ 讓模型懂你的領域：先試提示詞 → 不夠加 RAG → 最後才考慮微調
- 6 Temperature 只調取樣變異；可靠性還需依據、工具、拒答、評測與覆核

M7 名詞速查表 生成式 AI 基礎

(1 / 5)

中文	English	一句話說明
生成式模型	Generative Model	學習 $p(x)$ 或 $p(x,y)$ 等分布；LLM 依 context 預測下一 token
判別式模型	Discriminative Model	學習分界 $P(y x)$ ，回答「屬於哪一類」；M1–M3 學的都是這類
GAN	Generative Adversarial Network	生成器與鑑別器對抗學習；用於影像合成與合成資料
VAE	Variational Autoencoder	編碼到潛在空間再解碼；用於異常偵測與降維
擴散模型	Diffusion Model	從雜訊逐步去噪生成；目前影像生成主流
Transformer	Transformer	以注意力、MLP、殘差與位置資訊組成；現代 LLM 的主流基礎
注意力機制	Attention Mechanism	計算位置間的資訊權重；仍受位置表示、context 與模型偏誤影響
自注意力	Self-Attention	序列內位置彼此計算注意力；遮罩決定可看哪些位置
多頭注意力	Multi-Head Attention	並行多組注意力，可同時追蹤語法、語意等不同關係

M7 名詞速查表 生成式 AI 基礎

(2 / 5)

中文	English	一句話說明
Token	Token	模型處理文字的單位；字元、子詞與 token 的比例依 tokenizer 而異
子詞切分 / Tokenization	Tokenization	將文字切成字元 / 子詞 token 並映射為 ID；各模型切法不同
上下文長度	Context Window	單次能處理的 token 上限，含提示詞、召回段落與對話歷史
嵌入	Embedding	把文字轉成數值向量，語意相近者向量距離相近
餘弦相似度	Cosine Similarity	以向量夾角衡量相似度，RAG 召回排序的常用指標
向量資料庫	Vector Database	儲存嵌入向量並支援最近鄰搜尋的資料庫
Encoder-only	Encoder-only Architecture	以雙向注意力建立表示（如 BERT）；常用於理解、抽取與嵌入
Decoder-only	Decoder-only Architecture	用 causal mask 自回歸生成（如 GPT）；多數對話式 LLM 採用
Encoder-Decoder	Encoder-Decoder Architecture	序列轉序列架構（T5、BART）；用於翻譯與摘要

M7 名詞速查表 生成式 AI 基礎

(3 / 5)

中文	English	一句話說明
預訓練	Pretraining	以海量無標註語料訓練基礎模型；成本極高
微調	Fine-tuning	在既有模型上以領域資料再訓練，調整權重
推論	Inference	呼叫已訓練好的模型取得輸出；權重不變，按 token 計費
監督式微調	Supervised Fine-tuning, SFT	以標註的輸入輸出對進行微調
人類回饋強化學習	RLHF	以人類偏好排序優化模型輸出
自監督學習	Self-Supervised Learning	從無標註資料自動產生訓練目標（如遮字預測）
領域持續預訓練	Domain-Adaptive Pretraining, DAPT	用未標註領域語料持續做語言模型訓練，強化術語與語體
參數量	Parameters	模型權重的數量，如 7B、70B；大致對應能力上限
混合專家	Mixture of Experts, MoE	僅活化部分參數以提升推理效率的架構

M7 名詞速查表 生成式 AI 基礎

(4 / 5)

中文	English	一句話說明
多模態	Multimodal	能處理文字以外的輸入，如影像、音訊
Temperature	Temperature	控制取樣隨機性；低溫可重現，高溫多樣
核採樣	Top-p / Nucleus Sampling	只從累積機率前 p 的候選中取樣
最大輸出長度	Max Tokens	限制單次輸出的 token 數
幻覺	Hallucination	產生無依據、不忠實或錯誤內容；風險依模型、context 與任務而變
Hugging Face	Hugging Face	開源模型與資料集的集散平台
模型卡	Model Card	模型的說明文件，載明訓練資料、用途、限制與授權
量化	Quantization	降低權重精度以縮小模型體積與記憶體需求
GGUF	GGUF	本地推論常見模型格式之一；需同時查看量化、來源與授權

M7 名詞速查表 生成式 AI 基礎

(5 / 5)

中文	English	一句話說明
Ollama	Ollama	在本機執行開源 LLM 的工具，資料不離開電腦
AGI	Artificial General Intelligence	對跨領域通用能力的概念性稱呼；定義與是否達成仍有爭議